

Módulo **products** del repo REST API

Este documento recorre el código bajo **src/products**: capa HTTP (controlador + DTOs con **class-validator**), servicio de aplicación, contrato de persistencia (**interfaz** + token de inyección), implementación **TypeORM**, y entidad con relación **N:1** hacia categorías.

Contexto del proyecto

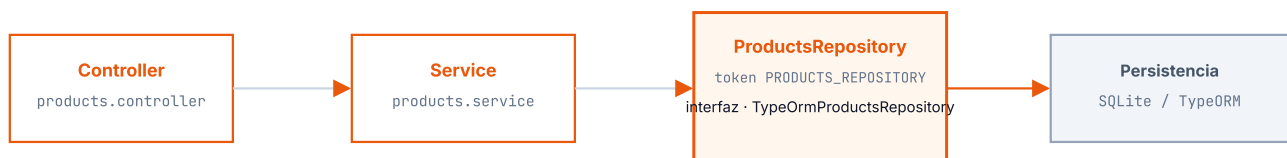
- **AppModule** registra SQLite (**database.sqlite**), entidades **ProductEntity** y **CategoryEntity**, y carga **ProductsModule**.
- **ProductsModule** es **@Global()**: exporta **ProductsService** y el token **PRODUCTS_REPOSITORY** para otros módulos.
- La validación de cuerpos asume un **ValidationPipe** global en **main.ts** (como en la otra presentación TypeORM).

Clic en **Bajar** abajo (o imprimir / guardar como PDF). Todo el contenido sigue en esta misma página.

VISTA DE CAPAS

De la petición HTTP al SQLite

El dominio expuesto al resto de la app es el tipo plano **Product** (**product.types.ts**). La entidad **ProductEntity** vive sólo en la capa de infraestructura; el repositorio la proyecta al tipo de dominio.



CategoriesService valida categoría antes de listar por categoría o crear/actualizar producto.

REQUEST → VALIDACIÓN DTO → SERVICIO → REPOSITORIO → BASE DE DATOS

Registro del módulo e inyección del repositorio

`TypeOrmModule.forFeature([ProductEntity])` habilita `@InjectRepository` en `TypeOrmProductsRepository`. El proveedor con `useClass: TypeOrmProductsRepository` satisface la interfaz detrás del string `PRODUCTS_REPOSITORY`.

```
@Global()
@Module({
  imports: [TypeOrmModule.forFeature([ProductEntity]), CategoriesModule],
  controllers: [ProductsController],
  providers: [
    ProductsService,
    { provide: PRODUCTS_REPOSITORY, useClass: TypeOrmProductsRepository },
  ],
  exports: [ProductsService, PRODUCTS_REPOSITORY],
})
export class ProductsModule {}
```

Entidad TypeORM y FK a categoría

Tabla **products**: clave entera autogenerada, nombre, precio (**real**), stock entero con default 0, **categoryId**. La relación **@ManyToOne** hacia **CategoryEntity** usa **onDelete: 'RESTRICT'**: no se puede borrar una categoría si sigue habiendo productos que la referencian (salvo que la base permita otra semántica).

```
@Entity('products')
export class ProductEntity {
  @PrimaryGeneratedColumn()
  id: number;

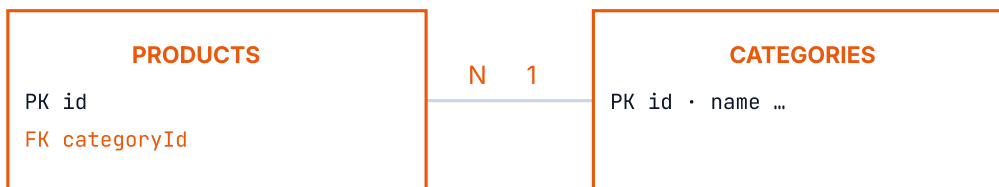
  @Column()
  name: string;

  @Column('real')
  price: number;

  @Column('int', { default: 0 })
  stock: number;

  @Column()
  categoryId: number;

  @ManyToOne(() => CategoryEntity, { onDelete: 'RESTRICT' })
  @JoinColumn({ name: 'categoryId' })
  category: CategoryEntity;
}
```



Varios productos comparten una categoría · RESTRICT al borrar categoría.

RELACIÓN VARIOS-PRODUCTOS / UNA-CATEGORÍA

product.types.ts y ProductsRepository

El tipo **Product** es el objeto que devuelve la API. **CreateProductInput** y **UpdateProductInput** son los shapes que entiende el repositorio (parciales en update). La interfaz **ProductsRepository** centraliza operaciones: listado paginado con filtros, CRUD, **updateStock** y **countByCategoryId** (usado por el módulo de categorías).

TIPOS

```
export type Product = {
  id: number;
  name: string;
  price: number;
  stock: number;
  categoryId: number;
};

export type CreateProductInput = { ... };
export type UpdateProductInput = { name?: string; price?: number; ... };
```

CONSULTA DE LISTADO · PRODUCTLISTQUERY

```
export type ProductListQuery = PaginationQuery & {
  name?: string;
  orderBy?: 'price' | 'name';
  order?: 'asc' | 'desc';
  categoryId?: number;
};
```

La paginación viene de **common/pagination.types** (**page**, **limit**, normalización y **toPaginatedResult**).

Orquestación y reglas de negocio

El servicio inyecta `PRODUCTS_REPOSITORY` y `CategoriesService`. Antes de `findAll` con `categoryId`, llama `categoriesService.findOne` para fallar si la categoría no existe. Igual al crear o al actualizar si cambia categoría: `ensureCategoryExistsForProduct`.

- `findOne / update / remove`: si el repo devuelve vacío → `NotFoundException`.
- `reduceStock`: si `quantity` supera `stock` → `BadRequestException('Stock insuficiente')`; luego `updateStock` en el repo.
- **Create**: mapea `CreateProductDto` → `CreateProductInput` antes de persistir.

Rutas REST y orden de declaración

Prefijo global `products`. Los query params de listado (`name`, `orderBy`, `order`, `page`, `limit`, `categoryId`) se parsean a mano: sólo se aceptan `orderBy price` o `name`, y `order asc` o `desc`; el resto se ignora en el objeto `ProductListQuery`.

Importante en Nest: la ruta `PATCH :id/stock` está declarada **antes** que `GET :id`. Si se invirtiera el orden, una petición a `/products/3/stock` podría coincidir primero con `:id` = la cadena `"3/stock"` y romper el enrutado.

MÉTODO	RUTA	NOTAS
GET	<code>/products</code>	Listado paginado + filtros
PATCH	<code>/products/:id/stock</code>	Cuerpo <code>ReduceStockDto</code>
GET	<code>/products/:id</code>	Detalle por id numérico
POST	<code>/products</code>	<code>CreateProductDto</code>
PUT	<code>/products/:id</code>	<code>UpdateProductDto</code> parcial vía validación opcional
DELETE	<code>/products/:id</code>	Borrado

Entrada HTTP tipada y validada

Tres clases en `src/products/dto/`. Con `strictPropertyInitialization`, conviene usar **aserción definitiva** (`property!: type`) en propiedades que rellena el `ValidationPipe` y no el constructor.

CREATEPRODUCTDTO

```
@IsString() @MinLength(2) @MaxLength(100) name: string;  
@IsPositive() price: number;  
@IsInt() @Min(0) stock: number;  
@IsInt() @Min(1) categoryId: number;
```

UPDATEPRODUCTDTO

```
@IsOptional() en cada campo · mismas reglas cuando el campo viene en el JSON
```

Permite actualizaciones parciales: sólo se validan las claves presentes.

REDUCESTOCKDTO

```
@IsInt() @Min(1) quantity: number; // en TS estricto: quantity!: number;
```

Qué llevarse de este módulo

- **Separación:** DTO (HTTP) → tipos de dominio / input → entidad (persistencia).
- **Testeable:** interfaz + token permiten mockear el repositorio en tests.
- **Integridad:** categoría validada en servicio; FK y **RESTRICT** en entidad.
- **API:** listado rico en query params; PATCH de stock con validación explícita del cuerpo.